KOENIG

step forward



Data Classes

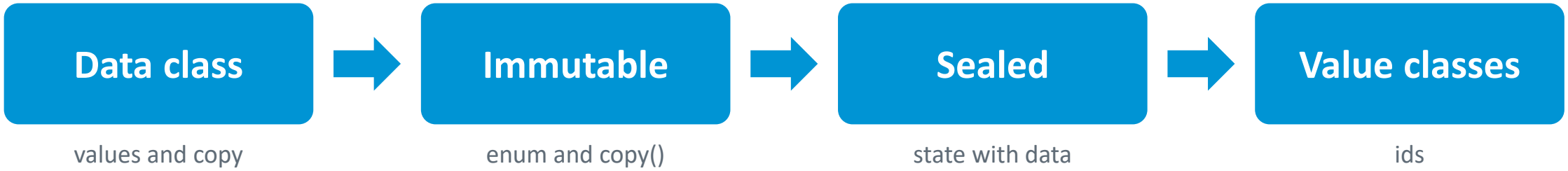
Kotlin for Java Developers

What You Will Learn

1. data class: equals, hashCode, toString, copy, componentN
2. Destructuring declarations and Enum classes
3. Immutability by default and the copy pattern
4. Sealed classes and interfaces for exhaustive state
5. Value classes and type-safe identifiers

Demo Scenario: ShopKart Order State

An order moves from placed to shipped to delivered, can be put on hold. We model it so that wrong states and mixed-up ids cannot even be written.



We will cut

- **Boilerplate**
- **Shared state**
- **Mixed-up ids**

Data classes

- **Keyword:** **data** before **class**.
- **Primary constructor:** needs at least one parameter.
- **Every parameter** must be val or var.
- **Not allowed:** abstract, open, sealed.
- **May implement** interfaces and have a body.

```
data class OrderLine(
    val sku: String,
    val name: String,
    val qty: Int,
    val unitPrice: Int
) {
    fun total() = qty * unitPrice
}

val line = OrderLine("P2", "Mouse", 2, 799)
```

Rule of thumb: a class whose job is to hold data

What the Compiler Generates

equals()
Structural equality: same values mean equal.

hashCode()
Consistent with equals, so sets and map keys work.

toString()
Readable: OrderLine(sku=P2, name=Mouse, ...).

copy()
New instance with some values changed.

componentN()
component1(), component2() ... for destructuring.

Only constructor properties count.
You may override any member.

equals, hashCode and toString

```
val a = OrderLine("P2", "Mouse", 2, 799)
val b = OrderLine("P2", "Mouse", 2, 799)

println(a)            // OrderLine(sku=P2, name=Mouse, ...)
println(a == b)       // true   equals()
println(a === b)      // false  same object?
println(a.hashCode() == b.hashCode())  // true
println(setOf(a, b).size)              // 1
```

Java: a.equals(b)
Kotlin: a == b

Java: a == b
Kotlin: a === b

== calls equals; === compares identity

Beware: Kotlin == is Java equals(), not Java ==

Only the Primary Constructor Counts

```
data class OrderLine(val sku: String, val qty: Int) {
    var note: String = ""   // body property
}

val a = OrderLine("P2", 2)
val b = OrderLine("P2", 2)
a.note = "gift wrap"

println(a == b)   // true: note ignored
println(a)         // OrderLine(sku=P2, qty=2)
```

- **Included:** constructor properties only.
- **Ignored:** properties declared in the body.
- **Use it** for derived values.
- **Avoid** var in data classes.

What is not in the constructor is not in equals

Demo: OrderLine as a Data Class

What we build

- **OrderLine:** one declaration, four fields.
- **== and ===:** equals against identity.
- **note:** a body property is ignored.

How the concept helps

- **Generated:** equals, hashCode, toString.
- **Sets:** equal lines count once.
- **Trap:** only constructor properties count.

componentN Functions

```
data class OrderLine(
    val sku: String,
    val name: String,
    val qty: Int
)

val l = OrderLine("P1", "Laptop", 1)
l.component1()   // "P1"
l.component2()   // "Laptop"
l.component3()   // 1
```

- **Numbered** in declaration order.
- **Generated** only for constructor properties.
- **Used by** destructuring declarations.
- **Normally** you never call them directly.

Reordering constructor parameters changes component numbers

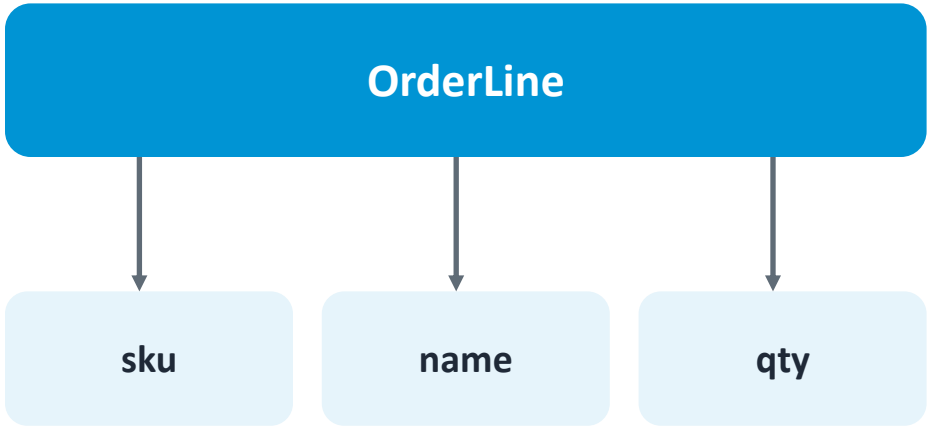
Destructuring Declarations

```
val line = OrderLine("P1", "Laptop", 1)

val (sku, name, qty) = line

val (_, title) = line          // skip with _
val (first) = line            // only the first

println("$sku: $name x$qty")
```



val (sku, name, qty) = line

Java: three getter calls. Kotlin: one line

Destructuring Beyond Data Classes

```
val (q, r) = 17 to 5              // Pair

val prices = mapOf("pen" to 20, "ink" to 70)
for ((item, price) in prices) {  // Map.Entry
    println("$item = $price")
}

val (big, small) = listOf(9, 2, 7).partition { it > 5 }
```

- **Works with** Pair, Triple, Map.Entry, partition.
- **In loops** and lambda parameters.
- **Needs** componentN functions.

Great for map loops: no entry.getKey() calls

Demo: Destructuring Declarations

What we build

- **component1():** what the compiler calls.
- **val (sku, name, qty):** one line.
- **Pair and map:** 17 to 5, for ((k, v)).

How the concept helps

- **By position:** not by name.
- **Loops:** no entry.getKey() calls.

Enum Classes: Fixed Constants

```
enum class OrderStatus {
    PLACED, SHIPPED, DELIVERED
}

val s = OrderStatus.PLACED
println(s.name)        // PLACED
println(s.ordinal)     // 0
println(OrderStatus.valueOf("DELIVERED"))
println(OrderStatus.entries)   // [PLACED, ...]
```

- **enum class** like Java enum.
- **name, ordinal** built in.
- **entries** preferred over values().
- **valueOf** throws on unknown name.

Same idea as Java, cleaner syntax

Immutability by Default

val

Read-only reference. Assigned once. Like a Java final local or field. Prefer it.

var

Mutable reference. Use only when the value truly has to change.

```
val title = "Report"
// title = "Other"      // error: val cannot be reassigned

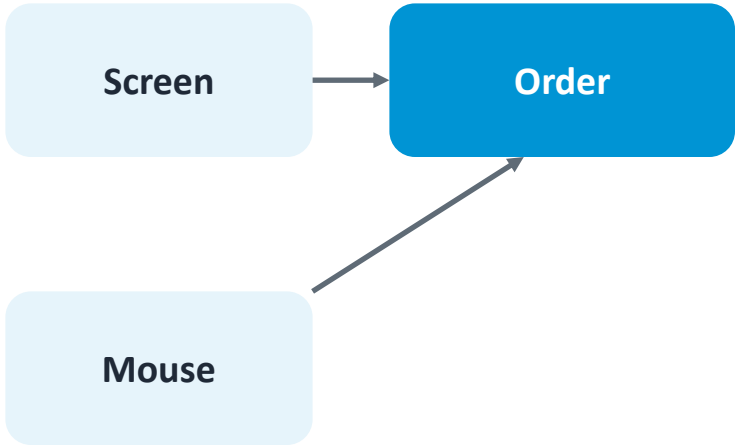
var counter = 0
counter++

val tags: List<String> = listOf("a", "b")   // read-only view
// tags.add("c")        // no add() on List
```

Start with val; change to var only if the compiler complains

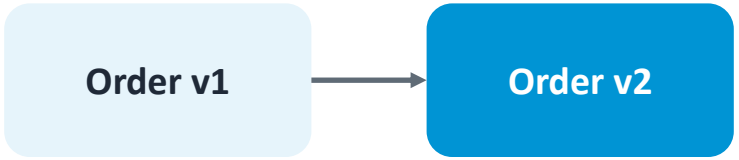
Why Immutable State?

Mutable: shared and changed



Who changed my order?

Immutable: new object per change



copy()

- **Safe sharing:** no one can change it behind your back.
- **Easy to test** and to reason about.

Immutable objects cannot be corrupted by other code

The copy() Pattern

```kotlin
data class Order(
    val id: String,
    val customer: String,
    val status: OrderStatus = OrderStatus.PLACED
)

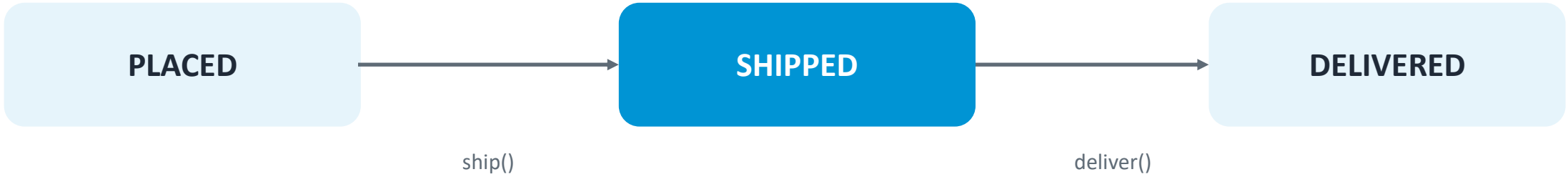
val original = Order("O1", "Asha")
val shipped  = original.copy(status = OrderStatus.SHIPPED)

println(original.status)  // PLACED
println(shipped.status)   // SHIPPED
println(original === shipped)  // false
```

- **copy()** takes the constructor parameters.
- **Defaults** are the current values.
- **Name only** what changes.

Java: builder. Kotlin: copy(field = value)

State Transitions with copy()




```
data class Order(val id: String, val status: OrderStatus) {
    fun ship() = copy(status = OrderStatus.SHIPPED)
    fun deliver() = copy(status = OrderStatus.DELIVERED)
}

val done = Order("O1", OrderStatus.PLACED).ship().deliver()
```

Each transition returns a new Order; the old one stays valid

Shallow Copy Warning

```
data class Basket(
    val owner: String,
    val items: MutableList<String>
)

val a = Basket("Asha", mutableListOf("Mouse"))
val b = a.copy(owner = "Bob")

b.items.add("Pen")
println(a.items)          // [Mouse, Pen]   shared!
```

- **copy()** copies references, not objects.
- **Mutable** contents are shared.
- **Fix:** use read-only List and build new ones.

```
val c = a.copy(items = a.items + "Pen")  // new list
```

Immutable properties make the shallow copy safe

Demo: Immutable Orders and copy()

What we build

- **List:** no add() on a read-only list.
- **Order:** copy(status = SHIPPED).
- **Basket:** the shallow copy trap, and its fix.

How the concept helps

- **Safe sharing:** old values stay valid.
- **Transitions:** ship() and deliver().
- **Fix:** read-only list plus new list.

When Enums Fall Short

Enum: one shape

Every constant has the same properties. SHIPPED cannot carry the tracking number.

Real states differ

Shipped needs tracking. Delivered needs a date. OnHold needs a reason.

```
// forced workaround with nullable fields
data class OrderRecord(
    val status: OrderStatus,
    val tracking: String?,          // only if SHIPPED
    val deliveredOn: LocalDate?   // only if DELIVERED
)
```

Nullable fields allow impossible states; sealed types do not

Sealed Classes and Interfaces

```
sealed interface Delivery {
    data object Placed : Delivery
    data class Shipped(val tracking: String) : Delivery
    data class OnHold(val reason: String) : Delivery
    data class Delivered(val on: LocalDate) : Delivery
}
```

- **sealed:** all direct subtypes are known at compile time.
- **Subtypes** live in the same package and module.
- **Each case** has its own data.
- **data object:** singleton with a readable toString.

Java: sealed / permits. Kotlin: sealed, no permits list

Exhaustive when

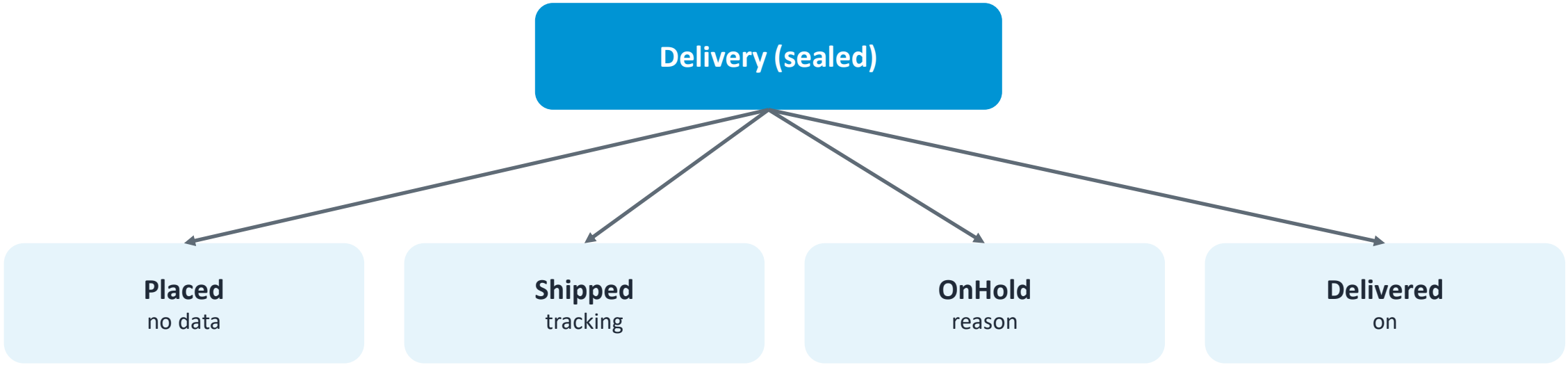
```
fun describe(state: Delivery): String = when (state) {
    is Delivery.Placed -> "waiting"
    is Delivery.Shipped -> "tracking ${state.tracking}"
    is Delivery.OnHold -> "on hold: ${state.reason}"
    is Delivery.Delivered -> "delivered ${state.on}"
    // no else
}
```

Add a new subtype
every when without else
stops compiling

With else
new case silently falls in
and fails at runtime

Leave out else: let the compiler find every place to update

Modelling State with Sealed Types



- **Impossible states** cannot be built.
- **Transitions use copy(delivery = ...).**
- **Pairs well with** immutable data classes.

sealed class, sealed interface, enum

enum class

Fixed constants. Same shape. Singleton instances. Best for simple labels.

sealed interface

Fixed set of types. Each case can have its own data. Subtypes can also implement other interfaces.

sealed class

Like sealed interface, plus shared state or constructor logic in the base class.

Default choice for state: sealed interface

Demo: Delivery as a Sealed Interface

What we build

- **OrderRecord:** an impossible state.
- **Delivery:** Placed, Shipped, OnHold, Delivered.
- **when:** exhaustive, no else.

How the concept helps

- **Right data:** each case carries its own.
- **Compiler:** finds every place to update.
- **Shipment:** copy() for each transition.

The Primitive Obsession Problem

```
fun addToCart(orderId: Long, productId: Long) { ... }

val orderId = 7L
val productId = 42L

addToCart(productId, orderId)     // compiles, silently wrong
```

- **Both are Long:** the compiler cannot tell them apart.
- **Bug found** in production, not at compile time.
- **Same for** email vs name (String).

Give every kind of identifier its own type

Value Classes

```
@JvmInline
value class OrderId(val value: Long)

@JvmInline
value class ProductId(val value: Long) {
    init { require(value > 0) }
}

val id = OrderId(7)
println(id)          // OrderId(value=7)
println(id.value)    // 7
```

- **One** val property in the constructor.
- **@JvmInline** is required on the JVM.
- **init** can validate.
- **Equality** by value, like data classes.

A new type with (almost) no runtime cost

Type-Safe Identifiers

```
fun addToCart(orderId: OrderId, productId: ProductId) { ... }

val orderId = OrderId(7)
val productId = ProductId(42)

addToCart(orderId, productId)       // OK
addToCart(productId, orderId)       // does not compile
addToCart(7, 42)                    // does not compile
```

error: argument type mismatch: actual type is 'ProductId', but 'OrderId' was expected.

The mix-up is now a compile error, not a data bug

Value Class Runtime Cost

Usually free

The compiler uses the underlying Long. Function parameters and fields hold a plain long.

Sometimes boxed

Nullable OrderId?, generics such as List<OrderId> and interface types allocate a wrapper object.

- **Still better** than a hand-written wrapper class.
- **Not for** classes with many fields: use data class.
- **Java sees** the raw Long in signatures (names are mangled).

Rule: one value, one meaning = value class

Demo: OrderId and ProductId Value Classes

What we build

- **Long, Long:** the swap compiles.
- **OrderId, ProductId:** @JvmInline, init, require.
- **addToCart:** the swap does not compile.

How the concept helps

- **Meaning:** one value, one type.
- **Cost:** usually the plain Long.
- **Validation:** in one place.

Key Takeaways

1. **data class**
   equals, hashCode, toString, copy, componentN for free.

2. **Destructuring**
   val (a, b) = x by position.

3. **Immutable first**
   val plus copy() for every transition.

4. **Sealed + when**
   Cases with data; no else; compiler checks.

5. **Value classes**
   OrderId and ProductId cannot be mixed up.



Happy Learning :)